SUMMARY

Polymorphism
The term comes from Greek and means "having many forms"
It allows related objects to be treated as the same kind of object in client code.

- A comon interface or abstract class is established
- Subclasses implement or override the necessary methods
- Code depends on the superclass instead of checking specific types

Advantages y Consecuences

Eliminates the need for multiple complex if-else statements.
Makes it easie to add new features or subclasses without modifying existing client code

Code becomes much shorter and cleaner

The exact behaviors is not know until the program runs, as it depends on the actual runtime class of the object

The abstraction is only the relevant parts of an object an ignoring unnecessary details

A general class is more abstract than its spec subclasses because it contain fewer details and concentrates the similarities shared among different class

Concepts in Java

Allows storing subclass objects in collection of the general superclass type
A superclass reference variable can hold an object of its subclasses (such), allowing generic collections to seamlessly handle various specialized subtypes

Overriding : Provides a new definition for an existing method inherited from a superclass, explicitly marked with the @override annotation for instance, a based subclass

Overloading : Defines multiple methods sharing the same name but with different parameter signatures within the same class, enabling methods to process varying inputs

VERZATIL

Escaneado con CamScanner

Abstraction

The act of extracting relevant details from a specific context while suppressing irrelevant ones. It involves sifting through large datasets to identify key concepts

An interface or definition that exposes essential public details while hiding internal implementation.

To ensure high abstraction quality in object-oriented programming, it must met the following criteria:

- Must be precise, of the right size, and use the correct gramatical category
- Names and labels must be clear and not unnecessarily redundant within their scope of use.
- All exposed aspects must be clearly declared in the code or its formal documentation

Avoid redundant mechanisms for the same functionality unless strictly necessary for backward compatibility or adapters

Interface

It is a structural component that defines a set of methods and constants without providing their full implementation

It acts as a reusable contract any class that chooses to implemented an interfaces is forced to provide the code for the methods declared in it using the implements

- Define pure behavior as a standard that different classes can adopt
Allow simulating multiple behavioral inheritance, as a class can implement multiple interface

By programming against interfaces rather than concrete implementations it is much easier to isolate components for unit testing

Helps comply whit the principle of hinding unnecessary design details, preventing issues like leaky abstractions

Escaneado con CamScanner